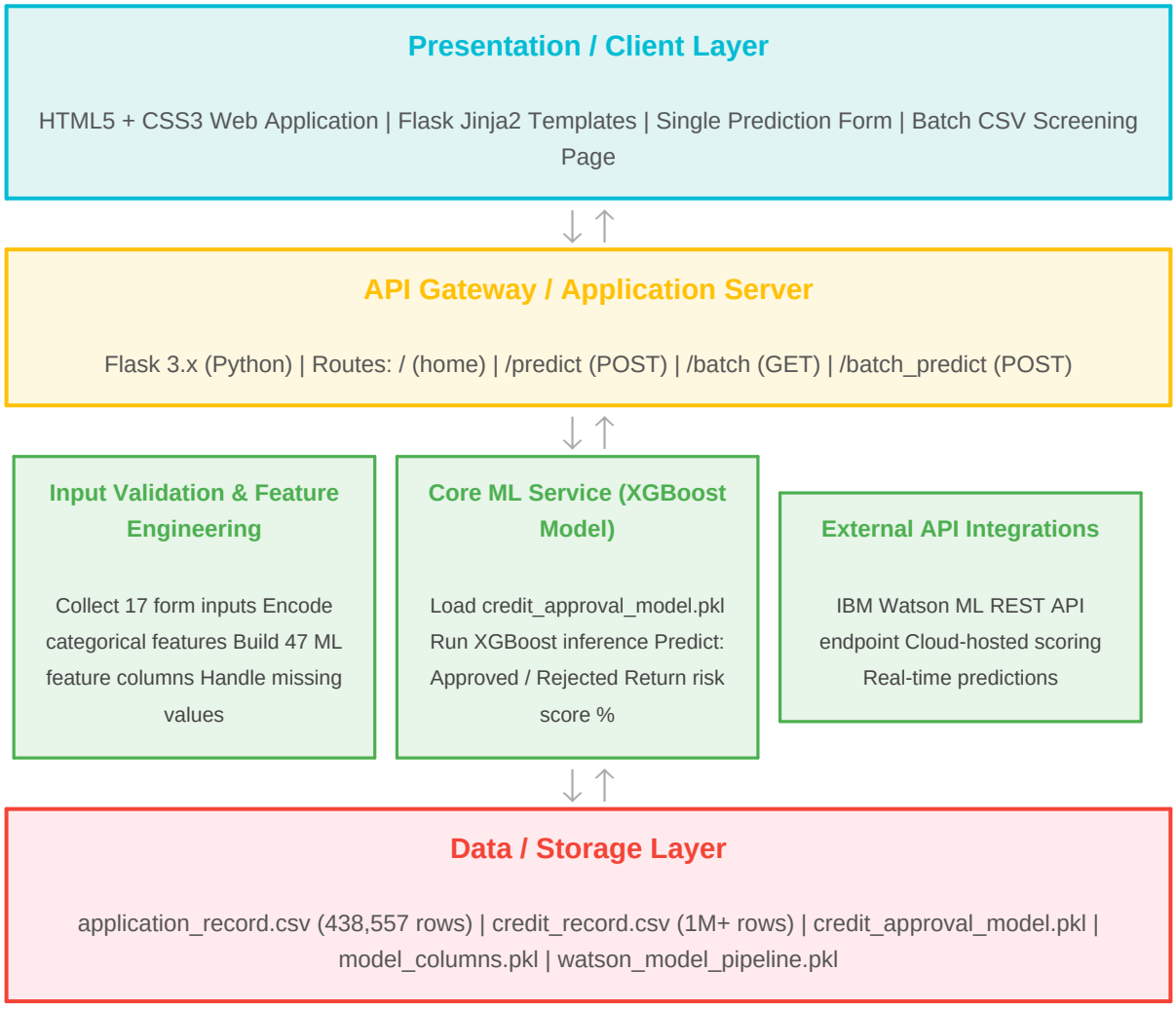
Solution Architecture

Date	03 July 2026
Team ID	
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Solution Architecture Diagram:

Provide a high-level visual representation of your solution's architecture. The diagram below maps out the Presentation Layer, Application Layer, and Data Layer with the specific technologies and components used in the Credit Card Approval Prediction project.



Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	The user-facing web interface where bank credit analysts, compliance officers, and prospective applicants interact with the system. Provides a single prediction form and a batch CSV upload page.	HTML5, CSS3, Jinja2 Templates, Flask
API Gateway / Application Server	Flask acts as the web server and API gateway, receiving HTTP requests from the browser, routing them to the correct handler (/predict or /batch_predict), and returning prediction results as rendered HTML responses.	Python 3.14, Flask 3.x, Werkzeug
Input Validation & Feature Engineering	Processes raw form inputs (17 fields) into the 47 model-ready features required by the XGBoost classifier. Handles one-hot encoding, binary flag creation, IS_UNEMPLOYED detection, and column order alignment.	Python, Pandas, NumPy
Core ML Service (XGBoost Model)	The trained XGBoost classifier loaded from credit_approval_model.pkl. Receives the engineered feature DataFrame, runs inference, and returns a binary prediction (0=Approved, 1=Rejected) along with a risk probability score.	XGBoost, Scikit-learn, Joblib
External API Integrations	IBM Watson Machine Learning cloud deployment pipeline. The trained model is uploaded to Watson ML repository, deployed as an online scoring endpoint, and integrated with the Flask application to serve cloud-hosted real-time predictions.	IBM Watson ML, IBM Cloud, REST API
Data / Storage Layer	Stores all raw datasets (Kaggle CSVs), processed train/test splits, and serialized model artifacts. The application loads model files at startup and reads CSV files during batch screening. No persistent database is used — all predictions are stateless.	CSV Files, Joblib PKL Files, JSON Metadata